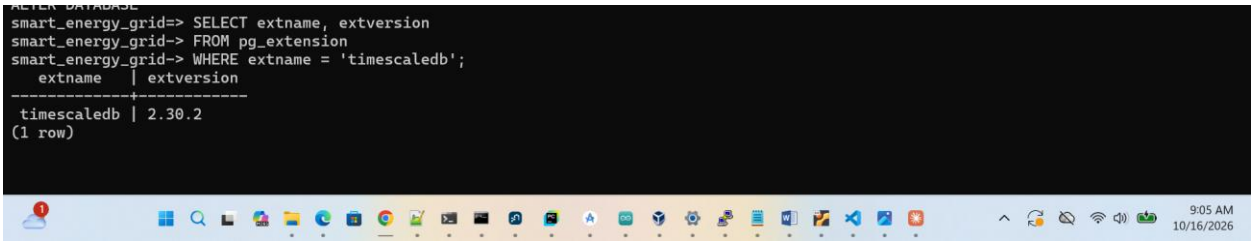
Seemore John

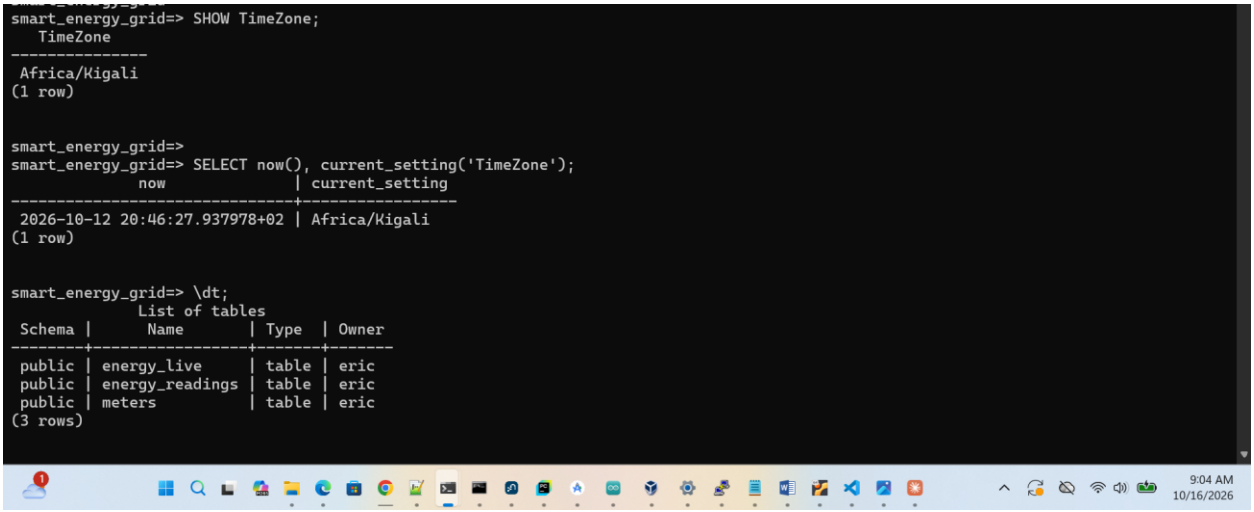
101408 DUSABIMANA Eric

101409 Odette DUSABEMARIYA

1.1 Confirm the Environment

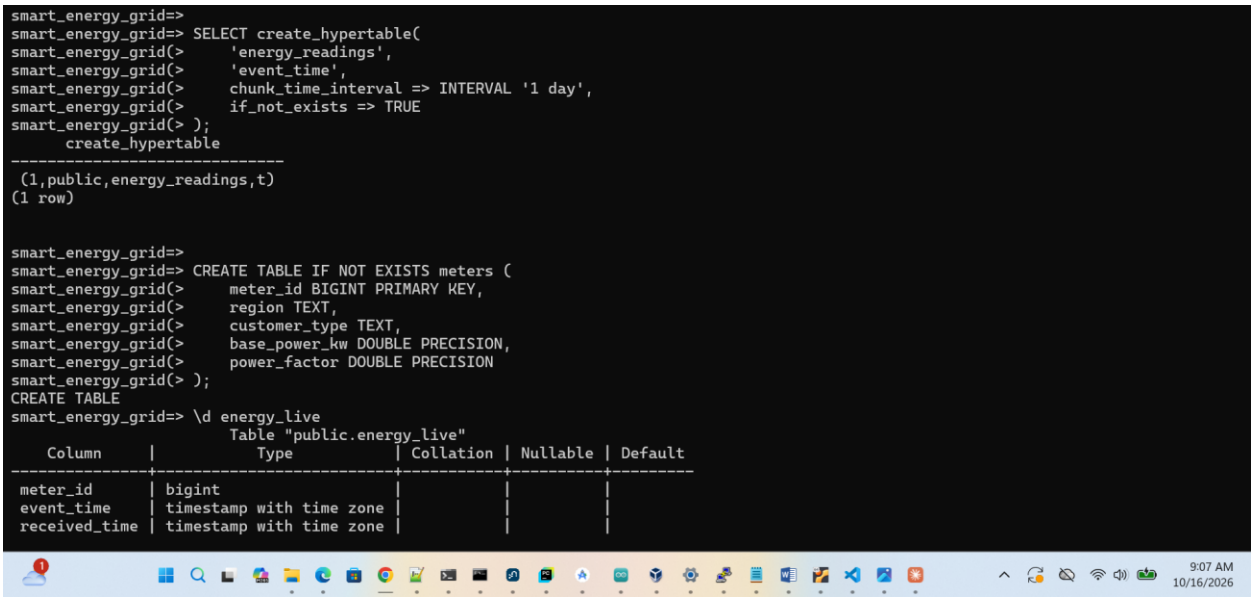
We enabled the TimescaleDB extension in the smart_energy_grid PostgreSQL database and verified that version 2.30.2 was active, meeting the assignment's minimum requirement of TimescaleDB 2.10 or later. We also configured the database-level time zone to Africa/Kigali to align the project with Central Africa Time (CAT). However, the subsequent session checks initially returned Etc/UTC, indicating that the current session had not applied the new setting. Later command outputs show timestamps with a +02 offset, demonstrating that CAT was being used in some subsequent queries. Consistent session configuration is important to ensure that daily aggregations, event timestamps, and reporting periods align with CAT midnight.

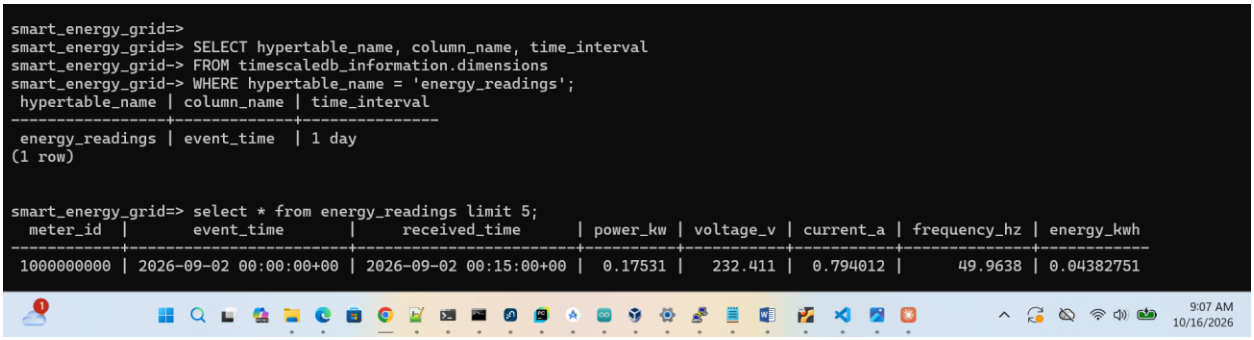




1.2 Create the Database Structures

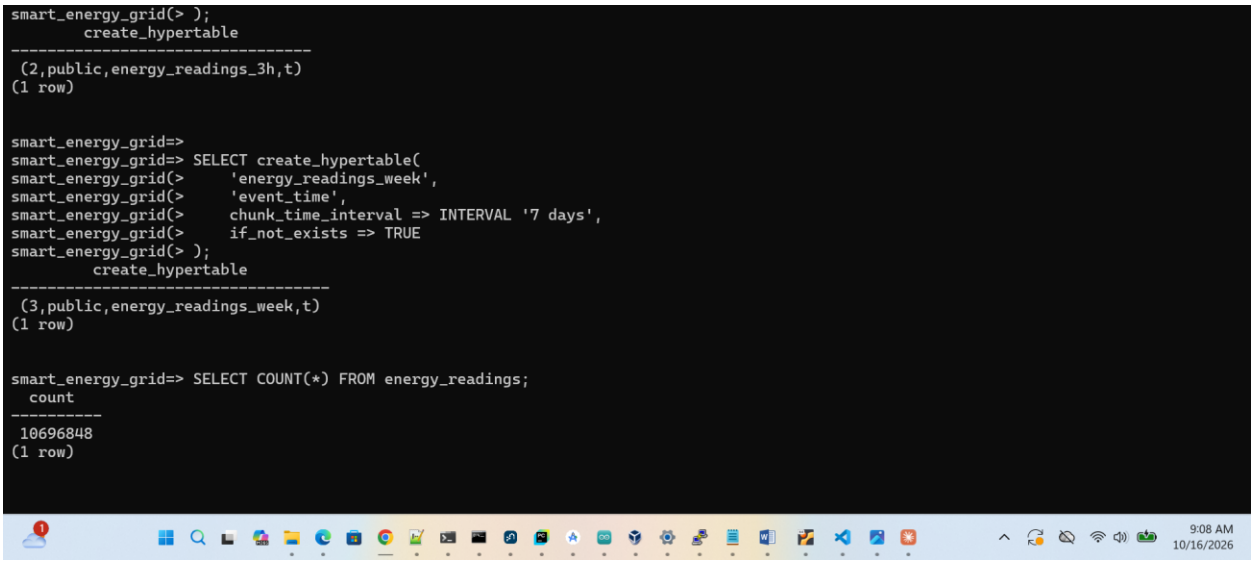
We created three PostgreSQL tables: energy_live for incoming smart meter data, energy_readings for historical measurements, and meters for storing meter information. The reading tables contain meter IDs, timestamps, and electrical measurements, while the meters table stores metadata such as region, customer type, and rated power. We also converted energy_readings into a TimescaleDB hypertable with a one-day chunk interval, improving the organization and efficiency of time-series queries and historical energy analys

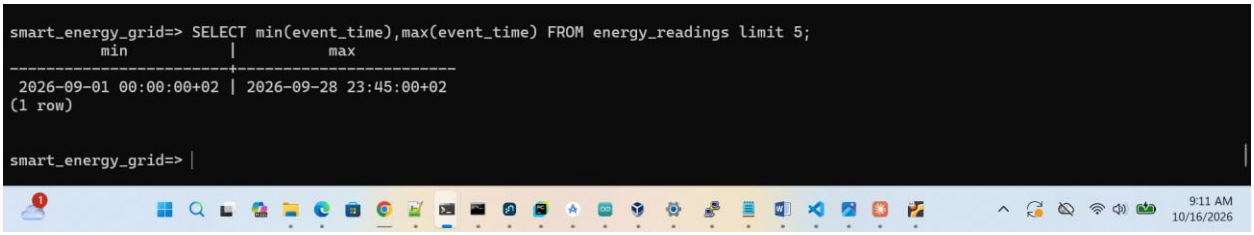




2.1 Capture and Load the Complete Stream

The database contains 10,696,848 energy readings collected from 4,000 distinct meters, covering the period from 1 September to 28 September 2026. The checks confirmed that there are no missing values in the energy_kwh column.The total number of rows matches the expected number of emitted readings, including possible duplicates. This confirms that the expected volume of data has been stored in the database.





2.2 Investigate Data Quality

The assignment requires identifying missing readings, duplicate copies, delayed arrivals, and completeness before cleaning. The expected reference values are on the screenshot where These reference values provide a benchmark against which the actual database results should be compared. The data-quality analysis must distinguish unique meter-and-time pairs from additional copies of those pairs. It must also identify delayed arrivals using received_time and calculate completeness, duplicate percentage, delayed percentage, and missing percentage.

2.3 Clean and Preserve a Consistent Benchmark

The data-cleaning process requires removing duplicate readings from **energy_readings** while retaining one valid record for each **meter_id** and **event_time** pair. Before deletion, measurement and arrival fields must be compared to identify identical duplicates and conflicting readings.After cleaning, the table should contain exactly 10,643,735 unique readings, with no duplicate pairs remaining. Missing time intervals must be preserved as gaps rather than replaced with zero consumption, since missing data does not necessarily indicate zero electricity usage.

3.1 Create and Verify Three Configurations

We created three TimescaleDB hypertables with different chunk intervals: one day, three hours, and seven days. Each table was populated with historical readings and indexed to support efficient time-based queries and meter-specific analysis.

The results confirmed that all three tables contained 10,696,848 readings from 4,000 meters, with matching timestamp ranges and consistent power and energy totals within the required tolerance. The tables contained 29, 225, and 5 chunks, respectively, demonstrating how different chunk intervals affect data partitioning and time-series query management.

3.2 Implementation and Time the Five Workloads

The command history includes several analytical SQL queries and execution plans using EXPLAIN (ANALYZE, BUFFERS, VERBOSE, SETTINGS) to evaluate PostgreSQL query performance. One example identifies the ten highest-demand 15-minute intervals between 22 and 28 September, with execution plans providing timing and buffer usage details.

However, further testing is required to meet the assignment requirements. Five defined queries must each be executed against all three hypertables, with one warm-up run followed by five measured runs per configuration. A complete performance report must include median, minimum, and maximum execution times, execution timestamps, and matching query results.

3.3 Explain the Chunk Results

The recorded chunk counts provide useful context for explaining query performance. The three-hour configuration has substantially more chunks than the daily or weekly configuration, while the weekly configuration has the fewest chunks. More chunks can increase planning and chunk-management overhead, whereas smaller chunks may help queries that filter narrow time ranges by limiting the amount of data accessed.

4.1 Create the Two Compressed Configurations

We created two additional hypertables, energy_time_compressed and energy_meter_compressed, using a one-day chunk interval. We configured both for TimescaleDB columnstore storage and ordered their data by event_time in descending order. The energy_time_compressed configuration uses time ordering without segmentation, while energy_meter_compressed uses the same time ordering with segmentation by meter_id. We also created indexes on event_time and (meter_id, event_time) for both tables and added columnstore policies targeting chunks older than 48 hours. These configurations establish two alternative storage strategies for evaluating the effects of time ordering and meter-based segmentation on storage efficiency and query performance.

4.2 Measure Storage and Compression Cost

The assignment requires comparing the baseline table's data, index, and total sizes against the corresponding sizes of both compressed tables. It also requires calculating compression ratios, percentages of storage saved, the number of compressed chunks, the percentage of rows held in compressed chunks, and the time required for compression operations.

4.3 Repeat the Query Comparison

The assignment requires rerunning all five benchmark queries against both compressed configurations and comparing their median execution times with the uncompressed table using the same chunk interval. The results should identify whether meter segmentation improves queries targeting a single meter and whether it affects queries covering the whole network.

A complete comparison requires consistent benchmark conditions, verified query results, and execution plans for both compressed tables. These results are not established by the uploaded command history.

5.1 Identify Demand Patterns

We executed a query to calculate daily network energy consumption by grouping readings into CAT-aligned daily buckets and summing energy_kwh. The query returned results for all 28 days from 1 to 28 September 2026. We also grouped the readings into four seven-day reporting periods and calculated total energy for each period.

The recorded weekly totals are on the screenshot, The results show a rising weekly energy total across the four reporting periods. This provides an initial indication of increasing aggregate consumption in the recorded dataset. The daily results also contain several noticeably lower totals, which should be investigated alongside missing-reading patterns before interpreting them as genuine reductions in demand.

We also executed a query grouping readings by event time and calculating network demand in kilowatts. This provides the data needed to identify the highest-demand 15-minute intervals.

5.2 Compare Regions and Customer Types

the assignment requires total energy, registered meter count, and energy per meter for every region and customer type across the full 28-day period. Those complete comparisons are not established in the available output.

a query joining energy_readings with meters using meter_id, then calculating regional energy totals using hourly buckets. This demonstrates how the readings can be associated with geographical metadata to investigate consumption across regions.

5.3 Assess Reporting Quality

The assignment requires calculating completeness for the network and each region, then comparing the energy total for 21 September using arrivals received by midnight on 22 September with the final total including delayed arrivals.

5.4 Investigate Abnormal Consumption

The command history shows attempts to query an anomaly_flags table, including meter identifiers, CAT event timestamps, power, energy, quartiles, interquartile range, and anomaly reasons. This indicates that an anomaly-analysis structure was being used to examine potentially unusual readings.

However, the database connection terminated unexpectedly while the query was running. Consequently, the available output does not establish the final list of flagged readings or the required evaluation against the 40 injected events. The report still needs to document the detection rule, thresholds, detected and missed events, false flags, and affected meters.

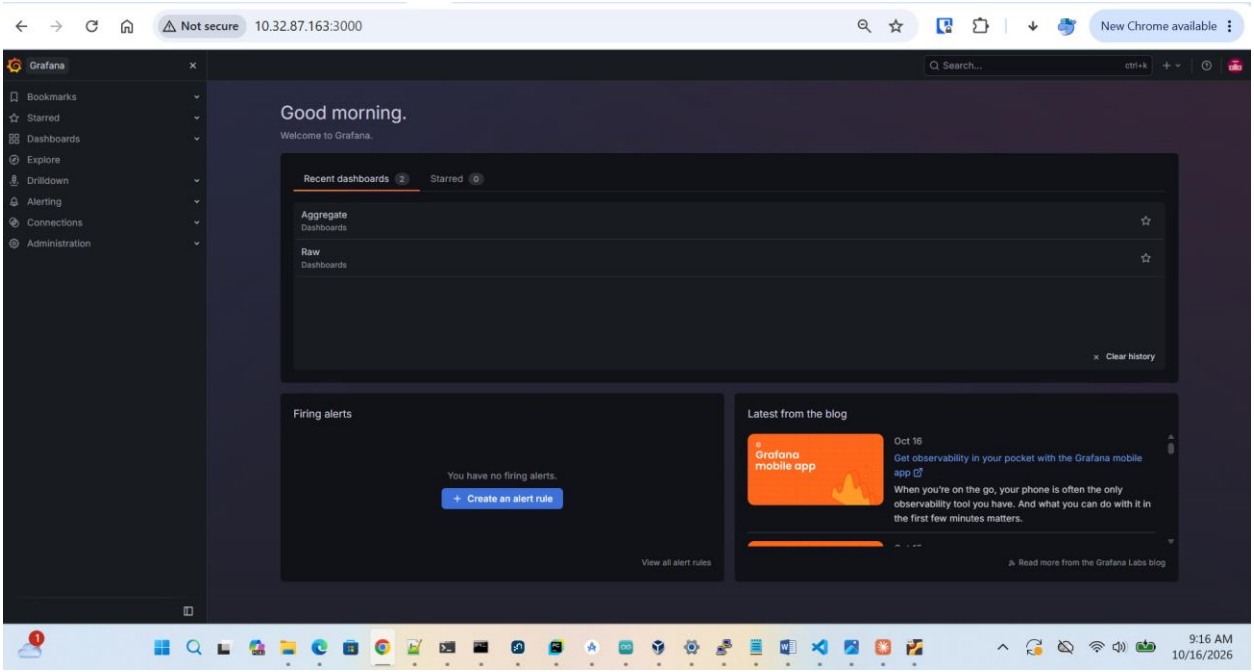
6.1 Create the Three Aggregate Views

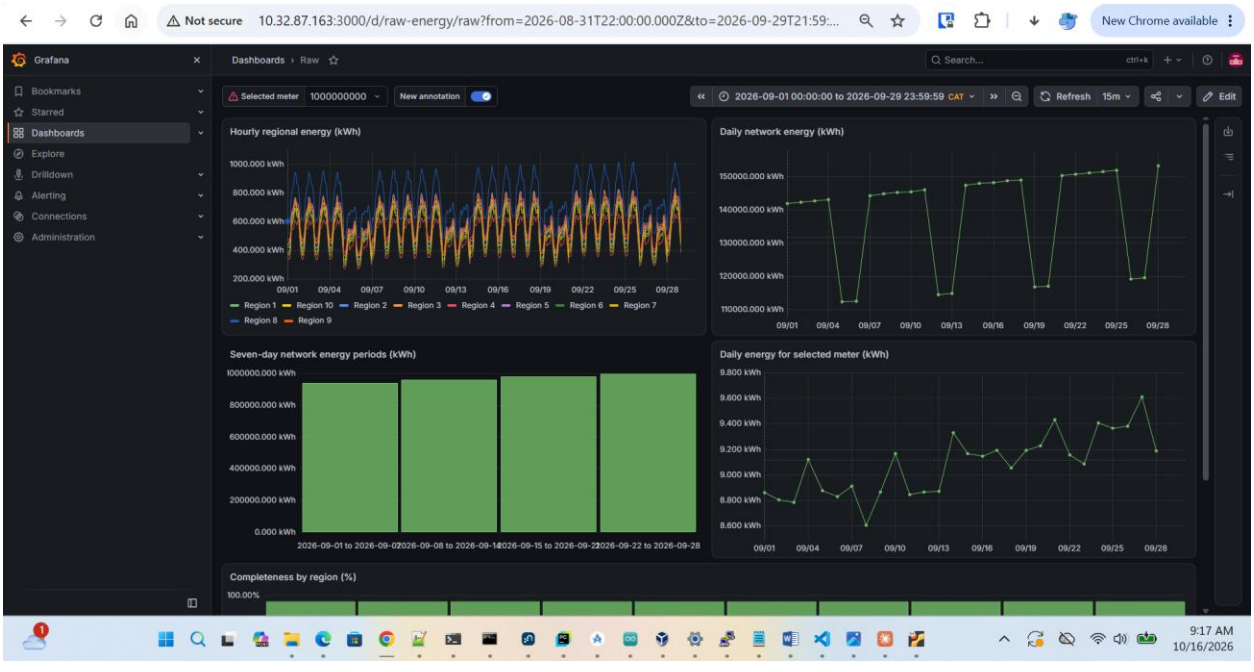
We created three TimescaleDB continuous aggregates to support energy analysis at different time intervals: 15-minute and hourly summaries by region, and daily summaries by meter. Each view calculates total energy, total power, and reading count, with regional views linked to the meters table through meter_id. Initially, the views were created empty and configured to display materialized data only. After refreshing, cagg_15m_region contained 26,880 rows, cagg_hour_region contained 6,720 rows, and cagg_day_meter contained 112,000 rows. These aggregates support detailed monitoring, regional reporting, and daily electricity consumption analysis.

6.2 Test Freshness and Delayed Arrivals

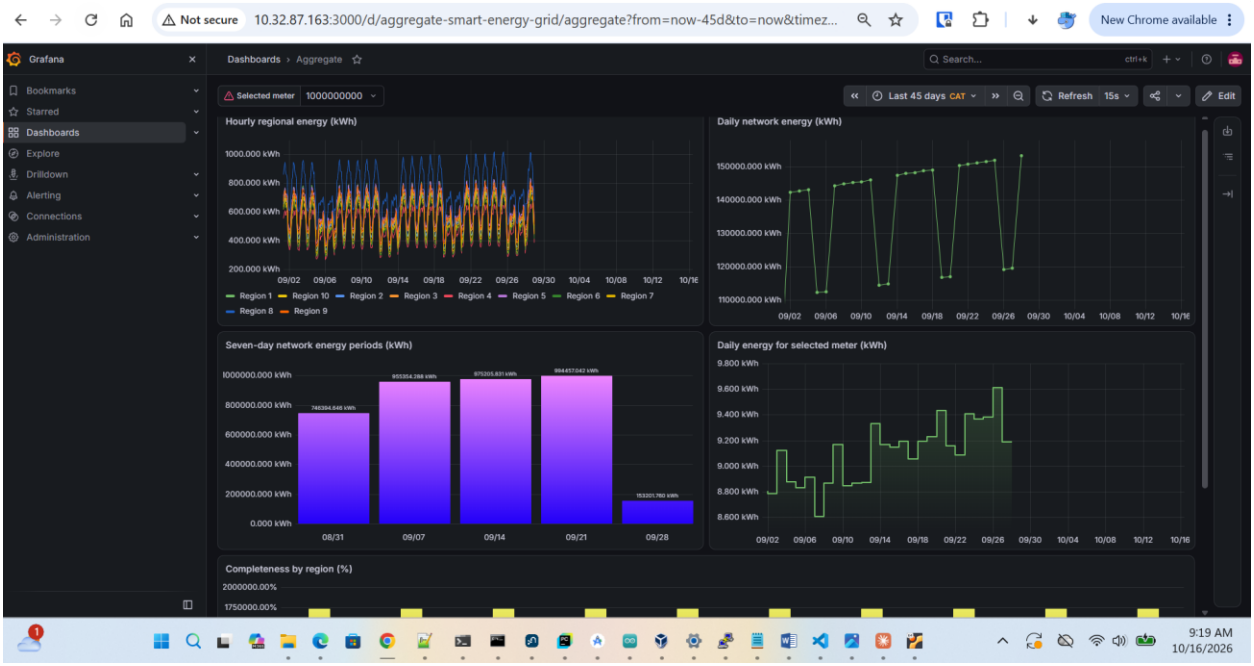
A separate freshness_test_readings hypertable and an initially empty hourly continuous aggregate, cagg_freshness_hourly, were created to test data freshness. The aggregate calculates total energy and reading counts for each CAT-aligned hour, with real-time aggregation initially disabled.A freshness_test_timeline table was also created to record test times and selected event timestamps. This setup provides a reproducible way to evaluate how manual refreshes affect aggregate results when new readings arrive for previously materialized and more recent time intervals.

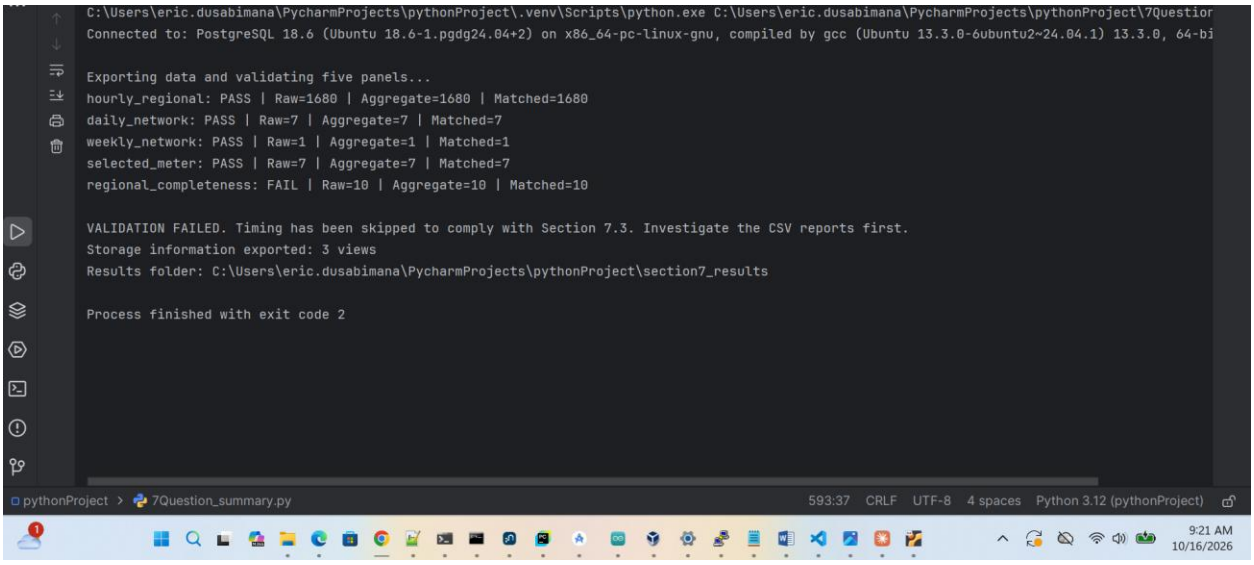
7. Use Grafana, Dash, or an equivalent tool. Build a dashboard named Raw whose measurement queries read energy_readings directly and join meters where needed. Use filters for whole CAT days and the selected meter.





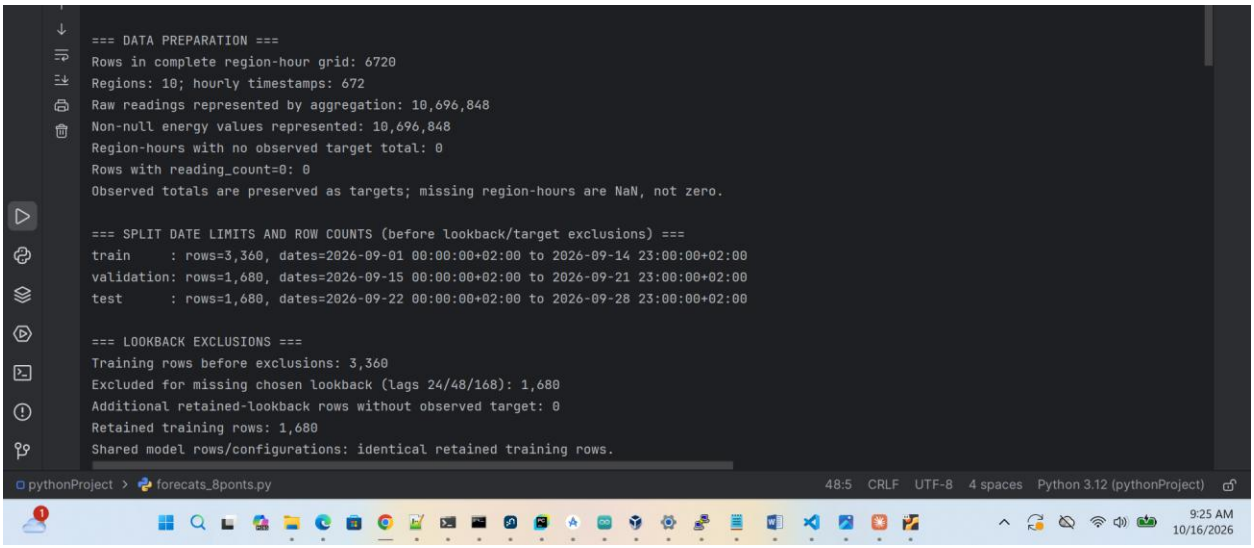
7.2 1. Build a second dashboard named Aggregate with the same five panels, layout, filters, and date range. Obtain hourly regional energy from the hourly regional view, daily and weekly network energy from the 15-minute regional view, meter history from the daily meter view, and observed completeness counts from the 15-minute regional view



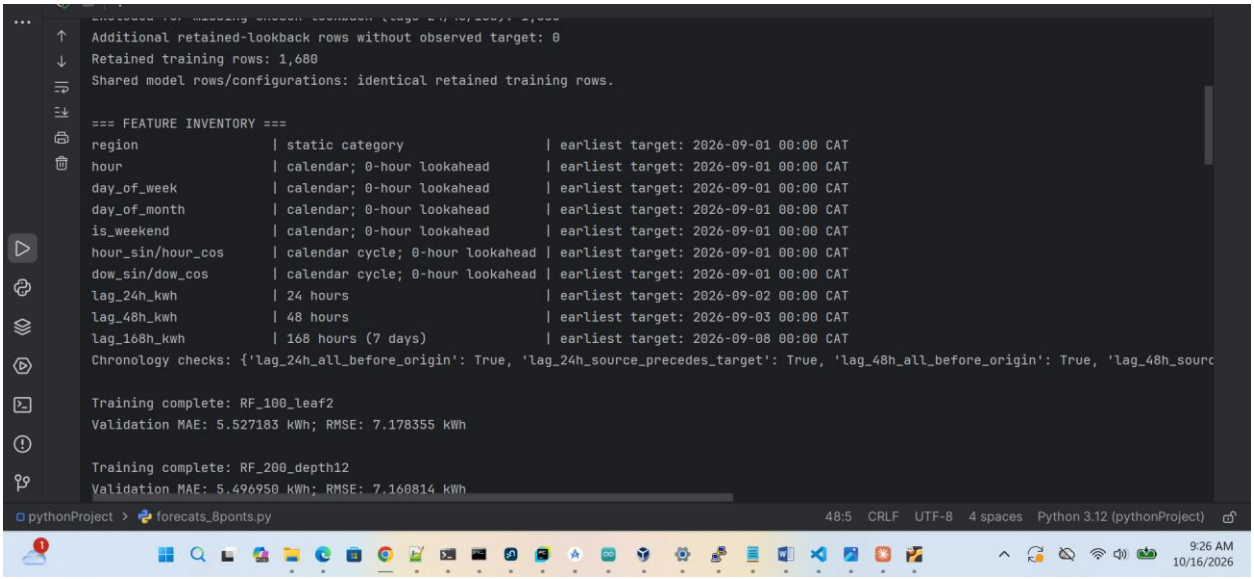


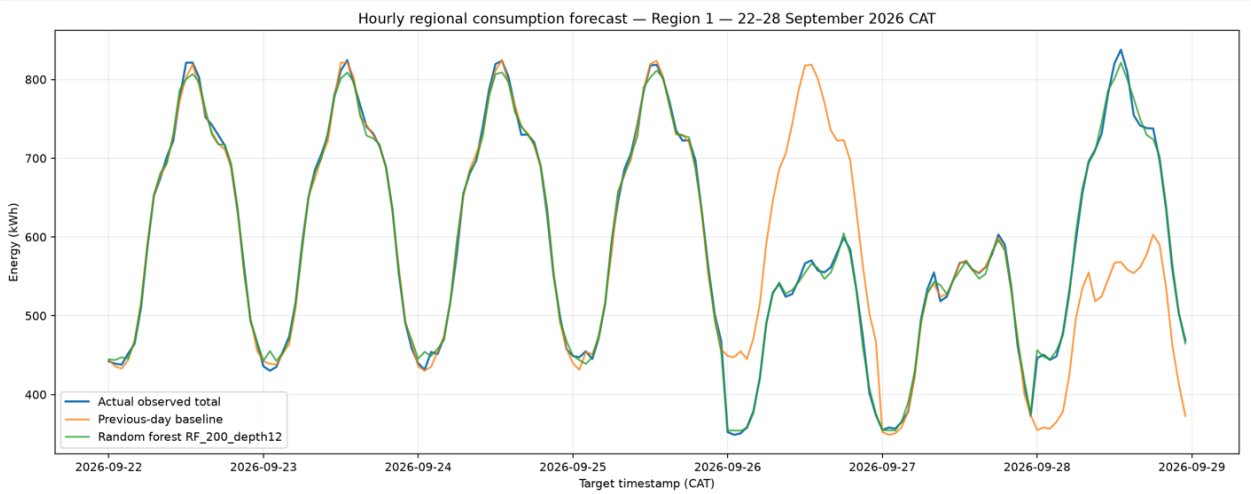
8. Forecast the next day

Create a Python analysis dataset with one row per region and CAT hour, summing energy_kwh from cleaned readings. Preserve observed totals as evaluation targets. Report reading counts and your handling of any absent hours or missing predictor values. 2. Split by target timestamp. Use 1 to 14 September for training, 15 to 21 September for validation, and 22 to 28 September for testing. Report row counts and date limits for each split. Fit preprocessing only on training data.



3. For each target hour, use 00:00 at the start of its day as the forecast origin. Build calendar features and lag features available before that origin. For example, the previous day's corresponding hour is available for every target hour. A preceding hour within the forecast day is not. Exclude training rows lacking the chosen initial lookback.

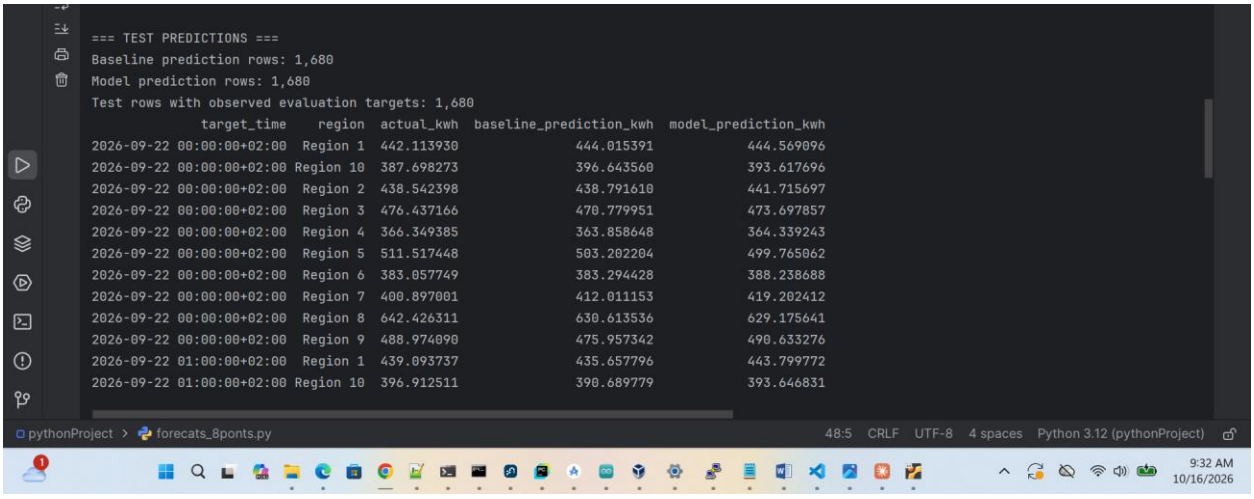




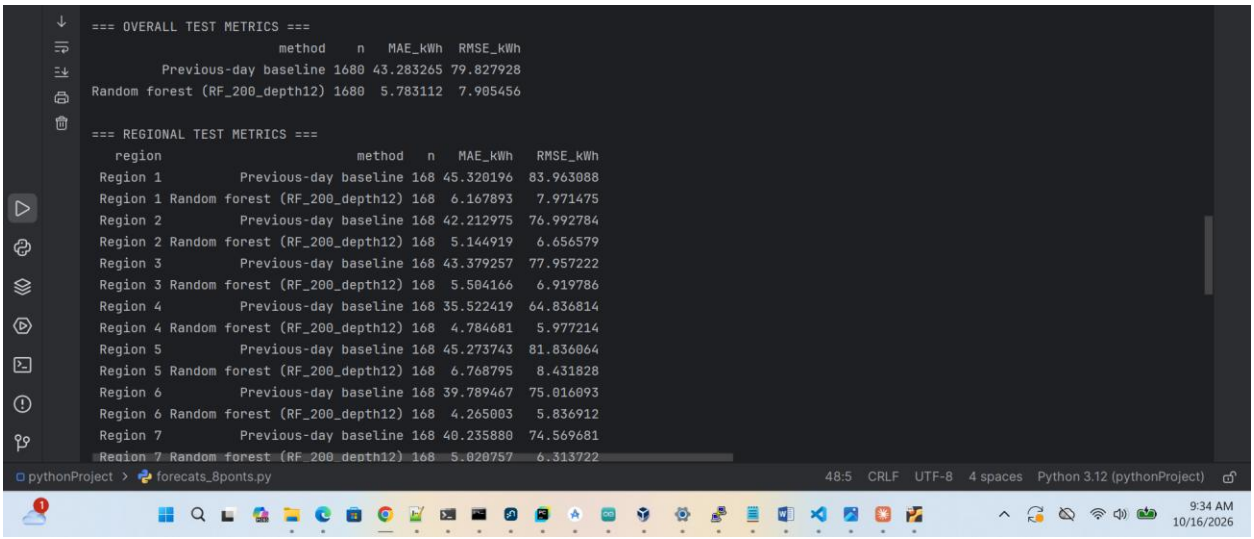
Name	Date modified	Type	Size
overall_metrics.csv	10/15/2026 5:31 PM	Excel.CSV	1 KB
regional_metrics.csv	10/15/2026 5:31 PM	Excel.CSV	2 KB
regional_peak_errors.csv	10/15/2026 5:31 PM	Excel.CSV	2 KB
seven_day_forecast_plot	10/15/2026 5:31 PM	PNG File	252 KB
test_predictions.csv	10/15/2026 5:31 PM	Excel.CSV	559 KB
validation_table.csv	10/15/2026 5:31 PM	Excel.CSV	1 KB

2. Plot actual, baseline, and model consumption for one region over all seven test days. For each region, find its highest actual test-hour consumption, breaking ties by earliest time. Report that hour, actual consumption, and each method's prediction and absolute error.

2. Generate the same 1,680 test predictions as the baseline, using only information available before each daily forecast origin. Earlier completed test days may supply later lag values but must not be used to refit the model or preprocessing.



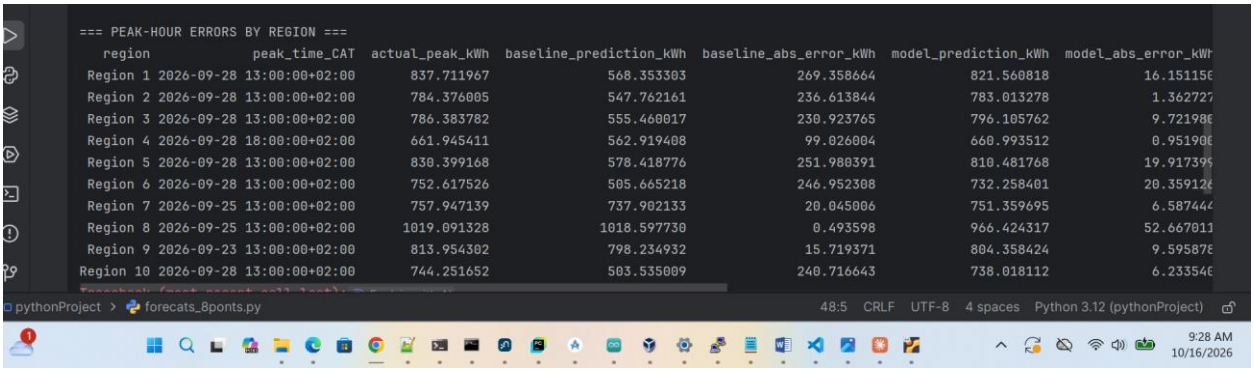
1. Evaluate a horizon of 24 hourly predictions per region per test day. Compare baseline and model MAE and RMSE over all 1,680 test observations and separately for each region. For prediction error use predicted minus actual kWh. MAE averages its absolute values. RMSE is the square root of its mean squared values. Both metrics are in kWh.



```
=== OVERALL TEST METRICS ===
                          method    n   MAE_kWh  RMSE_kWh
         Previous-day baseline 1680 43.283265 79.827928
Random forest (RF_200_depth12) 1680  5.783112  7.905456

=== REGIONAL TEST METRICS ===
   region                          method   n   MAE_kWh   RMSE_kWh
 Region 1          Previous-day baseline 168 45.320196  83.963088
 Region 1 Random forest (RF_200_depth12) 168  6.167893   7.971475
 Region 2          Previous-day baseline 168 42.212975  76.992784
 Region 2 Random forest (RF_200_depth12) 168  5.144919   6.656579
 Region 3          Previous-day baseline 168 43.379257  77.957222
 Region 3 Random forest (RF_200_depth12) 168  5.504166   6.919786
 Region 4          Previous-day baseline 168 35.522419  64.836814
 Region 4 Random forest (RF_200_depth12) 168  4.784681   5.977214
 Region 5          Previous-day baseline 168 45.273743  81.836064
 Region 5 Random forest (RF_200_depth12) 168  6.768795   8.431828
 Region 6          Previous-day baseline 168 39.789467  75.016093
 Region 6 Random forest (RF_200_depth12) 168  4.265003   5.836912
 Region 7          Previous-day baseline 168 40.235880  74.569681
 Region 7 Random forest (RF_200_depth12) 168  5.020757   6.313722
```


3. State whether the model improves on the baseline and whether its errors are useful for utility planning. Discuss peak errors, missing readings, and the effect of delayed arrivals on a live forecast.



```
=== PEAK-HOUR ERRORS BY REGION ===
   region            peak_time_CAT  actual_peak_kWh  baseline_prediction_kWh  baseline_abs_error_kWh  model_prediction_kWh  model_abs_error_kWh
 Region 1 2026-09-28 13:00:00+02:00       837.711967               568.353303              269.358664            821.560818            16.151150
 Region 2 2026-09-28 13:00:00+02:00       784.376005               547.762161              236.613844            783.013278             1.362727
 Region 3 2026-09-28 13:00:00+02:00       786.383782               555.460017              230.923765            796.105762             9.721980
 Region 4 2026-09-28 18:00:00+02:00       661.945411               562.919408               99.026004            660.993512             0.951908
 Region 5 2026-09-28 13:00:00+02:00       830.399168               578.418776              251.980391            810.481768            19.917399
 Region 6 2026-09-28 13:00:00+02:00       752.617526               505.665218              246.952308            732.258401            20.359126
 Region 7 2026-09-25 13:00:00+02:00       757.947139               737.902133               20.045006            751.359695             6.587444
 Region 8 2026-09-25 13:00:00+02:00      1019.091328              1018.597730                0.493598            966.424317            52.667011
 Region 9 2026-09-23 13:00:00+02:00       813.954302               798.234932               15.719371            804.358424             9.595878
Region 10 2026-09-28 13:00:00+02:00       744.251652               503.535009              240.716643            738.018112             6.233540
```